

Pygame Tug of War game fixes

10/08/2026 11:35:33

<https://claude.ai/chat/24ceac4f-ea9f-4368-b2ae-36882575a026>

I'm working on a Pygame Tug of War game (files: main.py,...

I fixed `game\_engine.py` as you described and sent it as a file. I used th...

Bug is fixed, thanks. Now Task 2: Dynamic AI panic surges. As the flag ge...

I added the panic surge to `game\_engine.py` as you specified. I sent yo...

Now Task 3: Rope tension & puller leaning animations. Implement exactl...

Rendering works (taut, humming rope; pullers leaning away from it)....

Last one, Task 4: Match timer and sudden death. In game_engine.py: - In...

Now test and write the README. I've updated `game\_engine.py` with th...



Used tool ▾

🔗 Set up for this session

👉 Done

10/8/2026, 11:08:57 AM



tug_of_war_original.zip

I'm working on a Pygame Tug of War game (files: main.py, game/game_engine.py, game/rope.py, game/player.py). The player pulls the rope left by alternating A and D keys; the computer pulls right every 180ms.

BUG: Rapidly mashing A and D drops inputs and can freeze input completely, so the computer always wins.

In GameEngine.handle_event, each pull sets is_pull_locked = True, and only the KEYUP of that same key clears it. Please:

1. Explain the root cause step by step (what happens when key presses overlap during fast mashing, e.g. D pressed before A is released, and what happens if a KEYUP is missed).
2. Fix it by removing the is_pull_locked mechanism entirely. A pull should register on every A/D KEYDOWN whose key differs from last_key (alternation still enforced, no dependency on KEYUP). Remove is_pull_locked from **init** and reset() too.
3. Add a short comment above the logic explaining why there's no lock.

Give me the full updated game_engine.py. Here is the code:
github link : https://github.com/SETAPESU26/56_tug_of_war

10/8/2026, 11:09:00 AM



- 🔑 Command failed
- 🔑 Ran a command
- 🔑 Read a file
- 🔑 Ran a command
- 🔑 Shared files

🕒 Done

I fixed `game_engine.py` as you described and sent it as a file. I used the version from your zip, not the GitHub link. I only checked that it compiles. I didn't run the game or test the mashing.

Root cause

The old logic was:

```
KEYDOWN (A or D): if not is_pull_locked and key != last_key → pull,
                  last_key = key, is_pull_locked = True
KEYUP:           if key == last_key → is_pull_locked = False
```

So the lock is set by one event (a successful KEYDOWN) and cleared only by a different event (the KEYUP of that same key).

1. **Normal slow play:** A down pulls and locks. A up unlocks. D down pulls and locks. D up unlocks. This works.
2. **Fast mashing:** Your fingers overlap, so D goes down before A comes up. At that moment `is_pull_locked` is still True from A, so the D press is ignored. It is dropped, not queued, and `last_key` stays A.
3. **A's KEYUP then arrives:** `last_key` is still A, so the lock clears. You now have to press D again, but D is already physically held down. Pygame doesn't repeat KEYDOWN for a held key (key repeat is off by default), so nothing happens until you lift D and press it again. This is the "dropped inputs" you saw.
4. **Missed KEYUP:** Suppose the lock is set by A, but A's KEYUP is lost (window focus change, OS key-event coalescing) or never matches `last_key`. Nothing else ever clears the lock, so every later KEYDOWN is rejected. That is the full input freeze. Because the computer keeps pulling every 180ms, it wins.

The fix

- `is_pull_locked` is removed from `__init__` and `reset()`.
- The KEYUP branch is removed, since nothing depends on it now.
- A pull registers on every A/D KEYDOWN whose key differs from `last_key`, so alternation is still enforced.

- I added a comment above the logic explaining why there's no lock.

```
# No pull lock: a lock released only by KEYUP drops presses  
when keys  
# overlap (D goes down before A is released) and can stay  
stuck forever  
# if a KEYUP is missed. Alternation is enforced purely by  
comparing  
# against last_key on each KEYDOWN, so input never depends on  
KEYUP.  
  
if event.type == pygame.KEYDOWN:  
    if event.key in (pygame.K_a, pygame.K_d):  
        if event.key != self.last_key:  
            self.rope.pull_left(1.0)  
            self.last_key = event.key
```

Nothing else in the project referenced `is_pull_locked`.

10/8/2026, 11:09:22 AM



Bug is fixed, thanks. Now Task 2: Dynamic AI panic surges.

As the flag gets pulled toward the player's winning line, the computer should panic and pull faster and harder. Implement it exactly like this in `game_engine.py`:

- In **init** add: `panic_threshold = 0.5`, `panic_min_cooldown = 130`, `panic_max_strength = 1.35`, `panic_level = 0.0`, with a comment saying it's tuned so a fast-mashing player can still win.
- Add method `compute_panic_level()`: `progress = (center - rope.marker_x) / (center - rope.left_win_x)` where `center = width / 2`. Return 0.0 if `progress <= panic_threshold`, otherwise ramp linearly to 1.0 at the goal (clamped with min).
- In `update()`: set `self.panic_level` each frame. Cooldown interpolates from `computer_pull_cooldown (180)` down to `panic_min_cooldown` based on `panic_level`. `Strength = random.uniform(0.7, 1.2) * (1 + (panic_max_strength - 1) * panic_level)`.
- In `render()`, while `PLAYING` and `panic_level > 0`, show a flashing orange (255, 140, 40) "COMPUTER PANIC SURGE!" with `font_small`, centered at `y = height - 60`. Blink interval = `int(400 - 250 * panic_level)` ms so it flashes faster as panic rises.
- Reset `panic_level` to 0.0 in `reset()`.

Give me the full updated `game_engine.py`.

10/8/2026, 11:11:50 AM



Read a file



Command failed



- 🔑 Ran a command
- 🔑 Shared files
- 👌 Done

I added the panic surge to `game_engine.py` as you specified. I sent you the full updated file.

I ran a headless check of the logic and the rendering. `compute_panic_level()` returns 0.0 at the centre and at the 50% mark, and 1.0 at the player's win line. `update()` and `render()` ran without errors, and `reset()` sets `panic_level` back to 0.0. I haven't played it, so the balance is untested.

Because `Rope.pull_right` uses `int(12 * strength)`, the pull size moves in whole pixels. At full panic the computer's pull ranges from about 11 to 19 px, and the interval drops from 180ms to 130ms.

10/8/2026, 11:12:17 AM



Now Task 3: Rope tension & puller leaning animations.
Implement exactly this:

game_engine.py:

- Track two decaying values: self.momentum (+ favours computer, - favours player) and self.struggle, with anim_decay = 0.93.
- Player pull: momentum -= 1, struggle += 1. Computer pull: momentum += strength, struggle += strength.
- New method update_animation(), called at the end of update(): multiply both by anim_decay; set rope.tension = min(1, struggle / 4); lean_bias = clamp(momentum / 2, -1, 1); player.lean_toward(0.2 + 0.8 * max(0, -lean_bias)); computer.lean_toward(0.2 + 0.8 * max(0, lean_bias)).
- Create pullers with facing=1 (player) and facing=-1 (computer). Render them with grip_y = rope.rope_y(puller.x + puller.facing * 28).
- reset() zeroes momentum, struggle, rope.tension and both lean values.

rope.py:

- Add rope_left_x = 60, rope_right_x = width - 60, tension = 0.0, max_sag = 18, max_vibration = 6.
- Add rope_y(x): u = clamped position along the rope (0..1), envelope = sin(pi * u), sag = max_sag * (1 - tension) * envelope, hum = max_vibration * tension * envelope * sin(x * 0.12 + ticks * 0.05). Return center_y + sag + hum.
- Draw the rope as pygame.draw.lines through points every 8px, width 10, colour (180+40t, 140+20t, 90-20t) where t = tension.
- The flag's y follows rope_y(marker_x).

player.py:

- Puller gets a facing param, lean = 0.0, MAX_LEAN_DEG = 28, LEAN_SMOOTHING = 0.15, and lean_toward(target) which eases lean toward the target.
- render(surface, grip_y=None): feet at (x, y+35); "up" vector tilts away from the rope by MAX_LEAN_DEG * lean. Draw the body as a 40x70 polygon along that vector, the head (radius 16) 85px up, and an arm line (width 6, skin colour) from shoulder (58px up) to the grip point (x + facing * 28, grip_y). Label position unchanged.

Give me all three full files.

10/8/2026, 11:12:47 AM



- 🔑 Ran a command
- 🕒 Modeling a character shape using geometric coordinates.
- 🔑 Ran a command
- 🔑 Read a file
- 👍 Done

Rendering works (taut, humming rope; pullers leaning away from it). Sending the files.

Used tool ▾

- 🔑 Shared files
- 👍 Done

I've updated all three files (`game_engine.py`, `rope.py`, `player.py`) as you specified. I ran the game headlessly with simulated A/D presses and the animation values moved as expected. A screenshot showed the rope taut and humming and both pullers leaning away from it. `reset()` zeroed everything. I haven't played it live.

- **Frozen at game over:** `update_animation()` is at the end of `update()`, so it stops once the state leaves PLAYING. The pullers and rope hold their last pose on the game-over screen.
- **Arms at rest:** The arm line goes from the shoulder to the grip point on the rope. With little lean the arm is nearly horizontal, which is what you described.

- `main.py` **unchanged:** No change was needed.

10/8/2026, 11:13:23 AM



Last one, Task 4: Match timer and sudden death. In `game_engine.py`:

- In **init**: `sudden_death_ms = 45_000`, `match_start = pygame.time.get_ticks()`, `elapsed_ms = 0`, `sudden_death = False`, `power_multiplier = 1.0`.
- In `update()`: `elapsed_ms = now - match_start` (only while PLAYING, so it freezes on Game Over). Once `elapsed_ms >= sudden_death_ms`, set `sudden_death = True` and `power_multiplier = 2.0`.
- Multiply the player's pull (and its momentum/struggle contribution) and the computer's strength by `power_multiplier`.
- Add `render_timer(screen)`, called right after the instruction text: "TIME MM:SS" centered at `y=12` with `font_small`. Grey (210,210,210) normally, yellow (255,200,60) in the final 10s before sudden death, red (255,70,70) during sudden death. During sudden death while PLAYING, also draw "SUDDEN DEATH x2 POWER" in red with `font_big` centered at `y=70`.
- `reset()` restores `match_start`, `elapsed_ms`, `sudden_death` and `power_multiplier`.

Give me the full `game_engine.py`. Then update `README.md`:

- Replace the wrong "Controls" line (it mentions HIGHER/LOWER buttons) with a table: A/D alternate = pull left, R = play again.
- Add a note under the install step: on Python 3.14+ use `pip install pygame-ce` if pygame fails to build.
- Add a "Changes Implemented" section with a summary table of the 4 tasks plus a short subsection per task

explaining what was changed (for Task 1, include the root cause).

10/8/2026, 11:14:37 AM



Fixing the timing logic for a sudden-death countdown feature. ▾

- 🔑 Ran a command
- 🕒 Fixing the timing logic for a sudden-death countdown feature.
- 🔑 Ran a command
- ✅ Done

Now test and write the README.

- 🔑 Ran a command
- 🔑 Read a file
- 🔑 Ran a command
- 🔑 Shared files
- ✅ Done

I've updated `game_engine.py` with the match timer and sudden death, and the `README.md` changes you listed. I tested the timer headlessly. It reads 00:00 in sudden death, the player's pull doubles from 12 px to 24 px, and `elapsed_ms` freezes at Game Over. `reset()` restores the timer state. I haven't played it live.

- **Timer direction:** You didn't say whether the timer counts up or down. I made it count down to sudden death, which fits the yellow warning in the final 10 seconds. It shows 00:00 once sudden death starts. If you'd rather it count up the elapsed time, that's a small change.
- **Banner overlap:** At the y positions you specified, the large "SUDDEN DEATH x2 POWER" banner (y=70) touches the instruction text (y=40). Moving the banner down to about y=90 would clear it.
- **README:**
 - The wrong controls line is now a table.
 - The Python 3.14+ `pygame-ce` note is under the install step.
 - The new "Changes Implemented" section has a summary table of the four tasks, plus a subsection per task. Task 1 includes the root cause.

10/8/2026, 11:15:13 AM